

The central design decision is that the agents do not communicate by hidden assumptions. They hand information through a shared ResearchState object.

Important shared state includes:

- conversation_id and turn_id
- user_question and conversation_history
- resolved_question and research_plan
- subqueries
- retrieved_chunks and ranked_chunks
- claims and verification_results
- verifier_verdict
- final_answer and citations
- latency_seconds, token_usage, retry_count, and LangSmith run information

3. The four agents — what each one actually does

Agent	Responsibility	Output
ResearchPlanner	Understands the question, resolves follow-ups, classifies the question, and researches subqueries	research plan
EvidenceRetriever	Searches ChromaDB for each subquery, merges results, retrieves relevant evidence.	retrieved evidence
EvidenceVerifier	Breaks the answer into claims and checks whether retrieved evidence supports them; detects conflicts using dates	verified evidence
GroundedSynthesizer	Writes the final answer using only verified evidence and general knowledge	final answer with citations or a grounded refusal.

3.1 ResearchPlanner

For a normal question, the planner can create one or more search queries. For a multi-hop question, it decomposes the task. For a follow-up such as “What about the Team tier?”, it uses conversation history to resolve what “Team” refers to.

3.2 EvidenceRetriever

The retriever is the tool-using agent. It calls the local ChromaDB vector search. The corpus metadata is preserved, including chunk_id, document ID, title, category, source URL, publication information, version, and text.

For multiple subqueries, the implementation uses Reciprocal Rank Fusion (RRF) so one subquery's results do not completely crowd out another subquery's evidence.

3.3 EvidenceVerifier

The verifier is the trust layer. It does not simply ask whether a document looks related. It checks atomic claims against the retrieved evidence. It can emit exactly four verdicts: supported, partially_supported, conflicting_evidence, and insufficient_evidence.

3.4 GroundedSynthesizer

The synthesizer receives the verified state and produces the user-facing answer. Citations are formatted as [chunk_id — title]. Unsupported questions are explicitly refused rather than answered from general knowledge.

4. How the RAG pipeline works

RAG means Retrieval-Augmented Generation. In this project, generation happens after evidence retrieval and verification rather than before them.

Step 1 — Ingestion

scripts/ingest.py reads corpus.jsonl, validates the records, creates local embeddings with sentence-transformers/all-MiniLM-L6-v2, and stores the vectors persistently in ChromaDB.

Step 2 — Question embedding

The user's question or planner-generated subquery is embedded with the same local embedding model.

Step 3 — Similarity search

ChromaDB returns semantically similar chunks. This is why the system can retrieve relevant text even when the user's wording differs from the document wording.

Step 4 — Multi-query merge

For multi-hop research, multiple searches are performed and their rankings are merged using RRF. The implementation deduplicates by chunk_id.

Step 5 — Verification

The verifier checks whether the evidence actually supports the claims and whether multiple documents disagree.

Step 6 — Generation

The synthesizer produces a grounded answer with exact source citations.

A useful sentence for a viva:

"The LLM is not the source of truth. The retrieved corpus evidence is the source of truth, and the LLM is used to reason over that evidence and express the verified result."

5. How difficult cases are handled

A major requirement was to behave differently when evidence is missing or contradictory.

Case A — Supported

If the retrieved chunks directly support the required claims, the verdict is supported and the answer includes the verified citations.

Case B — Partially supported

If some claims are supported but another required part is not established by the corpus, the system can mark the answer partially supported rather than presenting the whole answer as certain.

Case C — Conflicting evidence

The corpus intentionally contains historical/current differences. For example, one retention document states 90 days for cold storage while a newer version states 30 days. The verifier exposes both sources and compares their publication/version information. The synthesizer explains the conflict and identifies the newer/current value when the corpus supports that conclusion.

Case D — Insufficient evidence

For a question such as whether Kestrel has an official mobile SDK for Flutter or React Native, the system returns `insufficient_evidence`. It does not invent a citation or claim that such an SDK exists.

This is important because:

- Retrieval relevance is not the same thing as proof.
- A source mentioning a related concept does not automatically support the user's exact claim.
- The system makes uncertainty visible instead of hiding it inside a fluent answer.

6. Multi-turn conversation

The frontend supports multiple turns in a conversation. The backend maintains conversation history, and the ResearchPlanner uses that history to resolve references.

Example:

Turn 1:
User: What is the retention window for the Enterprise tier?

Turn 2:
User: What about the Team tier?

Planner resolves Turn 2 into:
What is the retention window for the Team tier?

Then:
resolved question → retrieval → verification → synthesis

The important point is that the second question is not treated as an isolated string. The planner carries the context from the previous turn into the new research plan.

The shared state contains `conversation_history` and `conversation_id`, so the workflow can preserve the context needed for follow-up resolution.

7. Backend and frontend

Backend

- FastAPI provides POST /api/chat for questions.
- GET /api/health reports service/model/tracing status.
- GET /api/conversation/{conversation_id} exposes conversation state/history.
- LangGraph executes the agent workflow.
- Pydantic Settings loads environment configuration.

Frontend

- React 18 + Vite + TypeScript + Tailwind CSS.
- Chat interface supports multi-turn questions.
- AgentStepper visibly shows Planner, Retriever, Verifier, and Synthesizer.
- VerdictBadge displays SUPPORTED / CONFLICTING / INSUFFICIENT style outcomes.
- EvidenceDrawer exposes retrieved evidence and metadata.
- Citation badges show the source chunk identifiers.

Why show the agent steps?

The assignment explicitly asks for behavior visibility. Showing the agent status makes the system easier to inspect and demonstrates that the answer is produced through an orchestrated workflow rather than a single opaque chat completion.

8. Why LangSmith was required

LangSmith is the observability layer for the agent workflow. The assignment requires reviewers to be able to follow a question through the workflow, including agent inputs/outputs and retrieval behavior, and to inspect evaluation runs.

In this project, LangGraph tracing is configured so a run can appear as a parent workflow with child runs such as planner, retriever, verifier, verifier decision, and synthesizer.

What you can show in a demo:

- A LangGraph parent run.
- Planner, retriever, verifier and synthesizer child runs.
- Inputs such as the user question and resolved question.
- Retrieved chunk IDs and subqueries.
- The verifier verdict.
- The final output.

Current access note: The LangSmith Developer plan page you encountered shows one seat and asks for a Plus upgrade for team collaboration. Do not claim that a reviewer was added as a member unless the invitation actually succeeds. Your traces themselves are useful evidence of observability, but reviewer access should be described accurately.

9. Evaluation — how you know it works

The project includes 20 original evaluation questions divided across five categories: single-hop, multi-hop, conflicting, unsupported, and follow-up.

The evaluation measures retrieval and answer quality rather than relying on a few demo screenshots.

Metric	Latest observed result	Meaning
Recall@5	0.7167	How often expected evidence appears within the top 5 retrieved chunks.
Precision@5	0.4300	How much of the top-5 retrieval set is relevant.
Faithfulness	1.0000	Whether generated claims stay grounded in the supplied evidence under the project's evaluation setup.
Relevance	0.8400	How relevant the response is to the question.
Correctness	0.8135	Agreement with the expected answer under the evaluation setup.
Citation Precision	0.7250	How often cited chunks actually support the answer.

The project also has a 9-test pytest suite, which passed during validation. The evaluation suite completed 20/20 questions after the ground-truth correction for q_single_03.

Concrete improvement

The initial multi-hop retrieval approach allowed dense similarity results from one subquery to crowd out evidence for another hop. The project changed retrieval to use RRF across subqueries and sharpened subquery decomposition.

Metric	Before	After
Multi-hop Recall@5	0.3541	0.4375
Multi-hop Precision@5	0.2500	0.3000
Overall Recall@5	0.7000	0.7167

10. Project structure — what each important file is for

Path	Purpose
corpus.jsonl	Original read-only Kestrel knowledge corpus.
scripts/ingest.py	Embeds corpus locally and populates ChromaDB.
backend/state.py	Shared ResearchState contract between agents.
backend/agents/planner.py	Question resolution, classification, subquery planning.
backend/agents/retriever.py	ChromaDB retrieval, multi-query merge, RRF.
backend/agents/verifier.py	Claim verification and conflict/insufficient-evidence logic.
backend/agents/synthesizer.py	Grounded answer generation and citations.
backend/graph/workflow.py	LangGraph orchestration and conditional retry flow.
backend/retrieval/	Local embedding and ChromaDB retrieval implementation.
backend/evaluation/metrics.py	Evaluation metrics.
backend/utls/citations.py	Citation parsing/validation.
backend/utls/retry.py	Bounded retry/backoff for Groq HTTP 429.
backend/main.py	FastAPI endpoints.
frontend/src/	React UI, API client, agent stepper, verdict/evidence/citation components.
scripts/evaluate.py	Runs the 20-question evaluation suite.
results/	Questions, per-question results, metrics, improvement report.
docs/	Architecture and engineering reflection.

11. Viva / interview questions — ready-to-say answers

Q1. Why did you use multiple agents?

“Because the task has different responsibilities. Planning, retrieval, verification and synthesis are easier to inspect and test separately. The shared state makes their handoff explicit.”

Q2. Why LangGraph?

“LangGraph gives me an explicit stateful workflow with nodes, edges and conditional routing. That fits this assignment because I need visible handoffs and a retry path when evidence is insufficient.”

Q3. Why ChromaDB?

“The assignment requires a local vector store. ChromaDB gives persistent local vector storage and similarity search without requiring a hosted vector database.”

Q4. Why local embeddings?

“Hosted embedding APIs were not allowed. I used sentence-transformers/all-MiniLM-L6-v2 locally, so the embedding step stays local.”

Q5. Why Groq?

“The assignment preferred a free-tier generation provider. Groq provides the generation model used by the project, with bounded retry handling for rate limits.”

Q6. What happens if the documents disagree?

“The verifier checks publication/version metadata and emits conflicting_evidence. The synthesizer shows both sources and explains which value is current when the corpus provides enough metadata to establish that.”

Q7. What happens if the answer isn't in the corpus?

“The system returns insufficient_evidence and explicitly says the available documentation is not enough. It does not hallucinate an answer or citation.”

Q8. How do citations work?

“Citations use the exact chunk ID and title, in the form [chunk_id — title]. The citation utility validates that the referenced chunk exists.”

Q9. What did you improve after evaluation?

“Multi-hop retrieval was weaker because one search could dominate the top results. I added RRF across subqueries and sharpened the planner's subqueries. Multi-hop Recall@5 improved from 0.3541 to 0.4375.”

Q10. What is the biggest limitation?

“The system's quality is bounded by the supplied corpus and retrieval quality. If the corpus lacks evidence, the correct behavior is to refuse rather than infer. Retrieval precision and citation precision also show room for improvement.”

12. 2-minute demo script

If someone asks you to demonstrate the project, use this sequence:

Step	Action	What to explain
1	Ask a normal question.	Show the answer, verdict and citations.
2	Open the agent trace.	Point to Planner → Retriever → Verifier → Synthesizer.
3	Ask a conflicting question.	Show both evidence sources and the conflicting_evidence verdict.
4	Ask an unsupported question.	Show insufficient_evidence and the absence of hallucinated citations.
5	Ask a follow-up question.	Show that the planner resolves the reference using conversation history.
6	Open LangSmith.	Show the parent LangGraph trace and child agent runs.
7	Mention evaluation.	20 questions, five categories, metrics, and the RRF improvement.

The key story is: plan → retrieve → verify → synthesize → cite.

13. Useful commands you should remember

From C:\Users\hp\Downloads\research-agent:

```
.\.venv\Scripts\python.exe scripts\ingest.py
.\.venv\Scripts\python.exe -m pytest tests/test_pipeline.py -v
.\.venv\Scripts\python.exe scripts\evaluate.py

# backend
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port 8000

# frontend
npm run dev --prefix frontend

# health check
Invoke-WebRequest http://127.0.0.1:8000/api/health -UseBasicParsing

# git
git status
git add .
git commit -m "... "
git push
```

If port 8000 says WinError 10048, that normally means another backend instance is already using port 8000. Check the health endpoint before starting another copy.

If Vite says port 3000 is in use and switches to 3001, that is also not a code failure; use the displayed localhost URL.

14. Final cheat sheet — memorize this

Architecture: React → FastAPI → LangGraph → 4 agents → ChromaDB/local embeddings → Groq generation.
Agents: Planner → Retriever → Verifier → Synthesizer.
Source of truth: retrieved corpus evidence, not the LLM's general knowledge.
Conflict handling: compare evidence + publication/version metadata → conflicting_evidence → explain both.
Unsupported handling: insufficient_evidence → grounded refusal.
Citations: [chunk_id — title].
Vector store: ChromaDB.
Embedding model: sentence-transformers/all-MiniLM-L6-v2 locally.
Generation: Groq llama-3.3-70b-versatile.
Orchestration: LangGraph StateGraph.
Evaluation: 20 questions / 5 categories / Recall / Precision / Faithfulness / Relevance / Correctness / Citation Precision.
Concrete improvement: RRF + sharper subqueries improved multi-hop Recall@5 from 0.3541 to 0.4375.

If you can explain those 12 lines naturally, you can explain the project without needing to read the code live.